Adevar Labs

Prepared for:

Pact Foundation

October 20, 2025

Pact Protocol on Aptos Audit Report

Version: Audit report after fix review

Table of Contents

1. Executive Summary **5**

About Pact Protocol 5

Audit Summary 5

Risk Profile 5

Overall Security Posture 5

Before Fix Review Summary 5

Improvement Recommendations 6

After Fix Review Summary 6

Audit Scope 6

2. Assumptions and Considerations **7**

Audit Assumptions 7

Centralization Risks 7

3. Severity Definitions **9**

Impact 9

Likelihood 9

Severity Classification Matrix 9

4. Findings **11**

M01: Missing fungible asset metadata validation enables funding with wrong/worthless asset 11

M02: Multiple funding allowed for pending loan leads to overwritten lender and stuck funds 12

M03: Griefing attack via unauthorized deposits to pending loan address 13

L01: Payment order bitmap allows ambiguous order 14

L02: Ungated Snapshot spam can bloat claim history 16

L03: DoS via unbounded vector growth and loops 17

L04: Late fee receiver locked to Originator by default 18

L05: Historical endpoints lack timestamp validation 19

L06: Incorrect verification of MAXIMUM_OBJECT_NESTING in ensure_owner::verify_descendant 20

L07: Incorrect calculation of aggregates::numerator_time_weighted_value() 22

L08: Forward function lacks only owner check 23

L09: PaymentCategorizerCreated event is not emitted correctly 24

L10: Unchecked input in set_late_fee_rules could lead to division by zero 26

L11: can_retire() is not implemented correctly leading to DoS in some cases 27

L12: Missing Validation on offer_loan() can DoS/Grief Borrowing 29

L13: Unrestricted payment schedule could lead to the creation of unrepayable loans 31

L14: Anyone can manipulate the loan.payment_count with 0 cost 33

L15: Attackers could grief last payment of the loan 35

L16: Possible spoofing in create_unnamed_escrow and create_unnamed_escrow_account 36

5. Enhancement Opportunities **38**

E01: Redundant whitelist object ownership check in whitelist::bulk_toggle 38

E02: Duplicate error code values creating ambiguous error handling 39

E03: Zero fee_numerator entries in FeeCollection are redundant 40

E04: Missing event emission in remove_late_fee_rules 41

E05: Redundant should_validate_due_date_continuity check 42

E06: Missing event emission in set_deferred_receiver 43

E07: Add event emission in whitelist::toggle 44

E08: Removing redundant check will improve gas efficiency 45

E09: Adjusting get_payment_schedule_summary_internal() will improve gas efficiency 46

E10: Unused DocumentSummaryAdded event 47

About Us **48**

About Adevar Labs 48

Audit Methodology 48

1. Program Context and Architecture Analysis 48

2. Threat Modeling 48

3. In-depth Manual Security Review 49

4. Detailed Fix Review and Validation 49

Confidentiality Notice 49

Legal Disclaimer 50

Appendix A: Appendix: Coverage 51

1. Executive Summary

About Pact Protocol

The Pact Protocol is a comprehensive on-chain lending infrastructure built on Aptos and written in Move, providing modular loan origination, management, and lifecycle tracking capabilities. The protocol implements NFT-based loan representation, advanced payment scheduling, custody support, and enrichment modules for specialized behaviors. It serves as a foundational DeFi infrastructure for creating sophisticated lending products on the Aptos blockchain.

Audit Summary

- Number of Findings: 19 total
    - Critical: 0
    - High: 0
    - Medium: 3
    - Low: 16
- Number of Enhancement Opportunities: 10

Risk Profile

The following table summarizes the distribution of identified vulnerabilities by risk level:

Risk Level	Count	Fixed	Acknowledged
Critical	0	0	0
High	0	0	0
Medium	3	3	0
Low	16	16	0

Overall Security Posture

The protocol demonstrates a well-architected and sophisticated lending infrastructure with strong foundational design. The audit identified several areas for improvement, primarily focused on enhancing validation mechanisms and optimizing data structure management. While most findings represent opportunities to strengthen the protocol's resilience against edge cases, the codebase shows thoughtful engineering and adherence to Move best practices. The identified issues are addressable through targeted improvements without requiring fundamental architectural changes.

Before Fix Review Summary

This initial audit report presents 19 findings for the team's consideration. We recommend prioritizing the Medium severity finding that enhance the protocol's robustness. Many of the Low severity findings and enhancements represent opportunities for gas optimization and improved monitoring capabilities.

Improvement Recommendations

Since the protocol is live on mainnet, we suggest a phased approach to implementing improvements:

1. **Near-term Monitoring**: Implement enhanced monitoring for the identified risk areas, particularly around loan funding flows and payment schedule processing
2. **Gradual Enhancements**: Consider implementing bounds checking for growing data structures in future upgrades to optimize gas costs
3. **Validation Improvements**: Strengthen input validation in new deployments or during regular maintenance windows
4. **Event Coverage**: Add supplementary events in future versions to improve observability without disrupting existing integrations
5. **Gas Optimization**: Apply the suggested optimizations during routine updates to improve user experience
6. **Continuous Testing**: Expand test coverage for edge cases identified in the audit to ensure ongoing protocol stability

After Fix Review Summary

The post-remediation review confirms that the Pact team fully addressed all 19 findings and delivered the requested enhancement work. The medium and the low severity items now include hardened validation and clearer error handling, closing the resilience gaps highlighted in the initial pass. The codebase has no outstanding issues or caveats.

Audit Scope

- **Repository:** https://github.com/Lucid-Fi/pact-aptos-initial-audit
- **Commit Hash:** **6894b8cc0fa377c85050b07c575d0bb59b88c580**
- **Files/Modules in Scope:**
    - core-loans
    - enrichers
    - utils
    - token-utils
    - deployment-utils
- **Exclusions:**
    - hybrid-book
    - test files

2. Assumptions and Considerations

Audit Assumptions

I. Aptos Framework Security: The audit assumes that the underlying Aptos blockchain and its core framework modules (AptosFramework, AptosTokenObjects) are secure and functioning as documented.

II. Move Language Semantics: The audit assumes Move language safety guarantees hold, including resource linearity, memory safety, and type safety enforced by the Move VM.

III. Deployment Configuration: The audit assumes proper deployment procedures will be followed, including correct address configuration, proper initialization of modules, and secure management of deployment keys.

IV. Time Source Reliability: The protocol heavily relies on **timestamp::now_microseconds()** for time-based calculations. The audit assumes the Aptos blockchain provides accurate and monotonically increasing timestamps.

V. External Token Standards: The audit assumes fungible assets and token objects used with the protocol conform to Aptos standards and behave predictably.

VI. Mathematical Operations: The audit assumes no integer overflow/underflow due to Move's built-in safety checks, though some division-by-zero risks were identified.

VII. Storage Cost Considerations: The audit does not deeply analyze gas costs or storage rent implications of unbounded data structure growth identified in findings.

Centralization Risks

I. Pending Loan Management: Funded but unstarted pending loans are not locked and rely entirely on admin intervention through **start_loan_with_ref()**. The protocol assumes administrators will act promptly and fairly to start these loans, but delays or selective enforcement could disadvantage certain participants.

II. Payment Schedule Modification Powers: Administrators can call **update_loan_payment_schedule_with_ref()** to completely overwrite existing payment schedules, including modifying principal amounts, interest rates, fees, and due dates for active loans. This gives admins the ability to fundamentally alter loan terms after origination.

III. Fee Structure Override: Through **fee_manager**, admins can modify origination and repayment fee structures at any time via **add_fee()** and **toggle_fee()**. These changes apply immediately and can impact borrowers mid-loan, with no grandfathering of existing loans.

IV. Late Fee Rules Modification: The **set_late_fee_rules()** function allows admins to change late fee calculation parameters (base amounts, percentages, grace periods) retroactively affecting all existing loans, not just new originations.

V. Payment Schedule Validation Toggle: Through **toggle_payment_schedule_principal_validation()**, admins can disable critical validation checks, potentially allowing creation of structurally unsound loans with discontinuous principal amounts.

VI. Whitelist Gatekeeping: The whitelist module (**whitelist::toggle()**) gives admins complete control over who can hold loan NFTs and participate as lenders, creating potential for selective access and censorship.

VII. Historical Payment Adjustments: Functions like **repay_loan_historical()**, **repay_loan_historical_with_seed()** allow admins to retroactively modify payment records and loan states, potentially rewriting loan history.

VIII. NFT Transfer Control: Through **nft_manager**, admins control transfer restrictions and can set deferred receivers, potentially redirecting loan ownership without holder consent.

IX. Fee Distribution Control: Late fees always route to the originator by default (via **get_late_fee_receiver()**), and there's no exposed function for borrowers to redirect these fees, creating permanent revenue streams controlled by loan originators.

X. Object Ownership Hierarchy: The entire deployment infrastructure relies on object ownership patterns that create cascading control, whoever controls the root deployment objects effectively controls all protocol upgrades and modifications.

Trust Assumptions

I. Off-Chain Document Verification: The document manager module stores hashes but assumes off-chain systems properly verify and store actual documents.

II. Risk Score Attestation: Risk scores are set by authorized attesters without on-chain validation of scoring methodology or accuracy.

III. Frontend Integration: Users must trust frontend applications to properly construct transactions and display loan terms accurately.

IV. Event Monitoring: The protocol assumes off-chain systems monitor events for tracking loan states, payments, and system health.

V. Deployment Scripts: Trust in deployment and configuration scripts to properly initialize the protocol with secure parameters.

VI. Cross-Module Integration: Trust that all protocol modules are deployed and configured to work together correctly, especially enrichers with core lending logic.

Considerations

I. Permissionless repay: The permissionless nature of the repay function allows anyone to make small or partial repayments on behalf of the borrower, potentially causing accounting mismatches or confusion in off-chain tracking systems.

3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions help project our team's understanding of both the potential consequences of a vulnerability and how likely a vulnerability is to be discovered and exploited in the real world.

Impact

Impact reflects the potential consequences of the issue–particularly on **project funds**, **user funds**, and the **availability or integrity** of the protocol.

- **High Impact**: Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact**: Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact**: The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

Likelihood

Likelihood reflects how easy a vulnerability is to discover and exploit by an attacker, as well as how economically attractive the exploit is to an attacker.

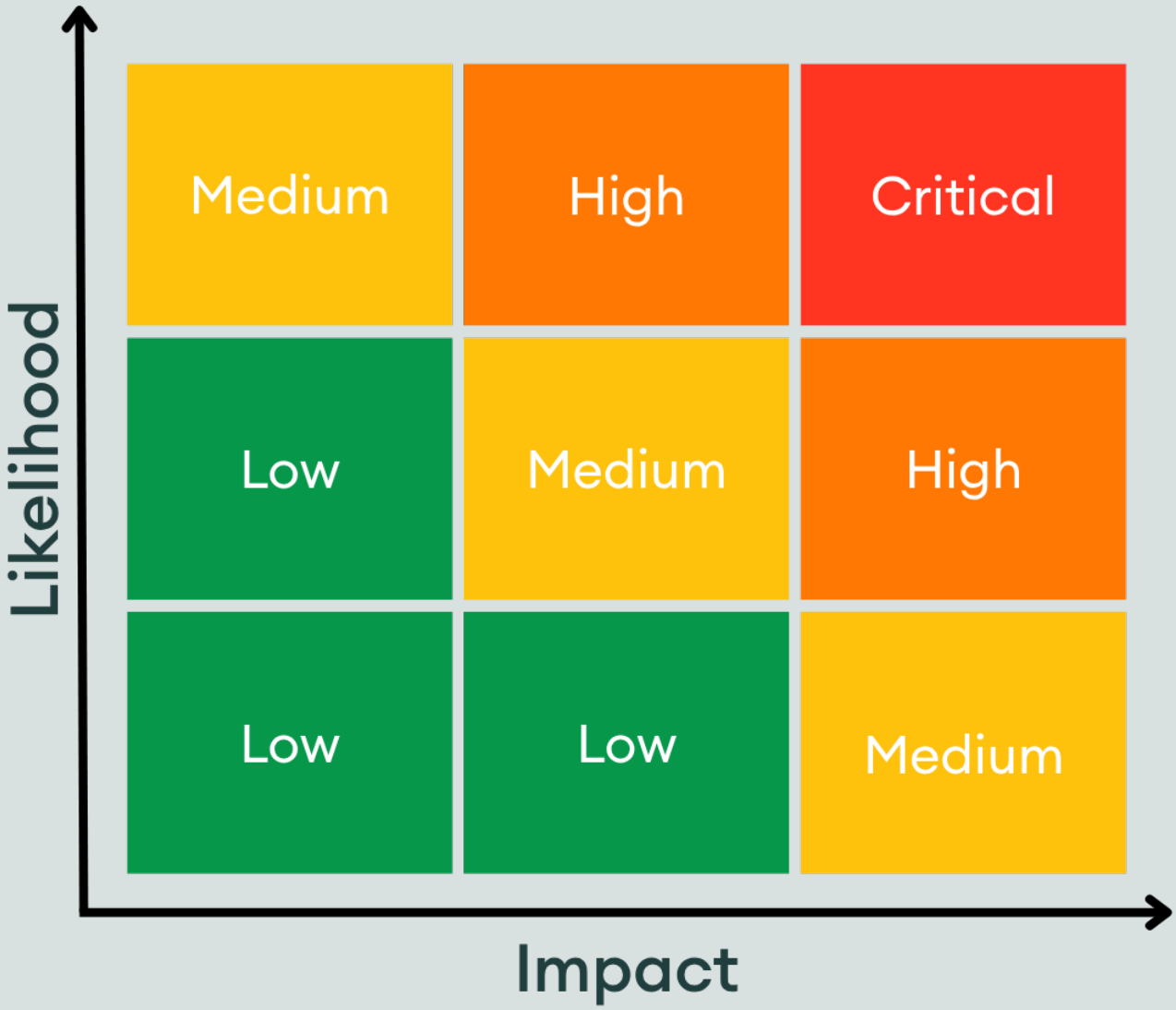
- **High Likelihood**: The vulnerability is trivially exploitable. This means it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risks.
- **Medium Likelihood**: This type of vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but with manageable financial risk.
- **Low Likelihood**: Exploitation of these vulnerabilities is often technically unfeasible or requires highly specialized conditions. They may require extraordinary effort or a significant financial risk for an attacker, with a high chance of failure and minimal potential return.

Severity Classification Matrix

By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

- **Critical**: High impact + high likelihood (e.g. a bug that could allow anyone to drain a substansial amount of protocol funds with minimal effort)
- **High**: High impact with medium likelihood, or medium impact with high likelihood
- **Medium**: Moderate impact and/or discoverability
- **Low**: Minimal impact or unlikely to be exploited

This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.



Severity Matrix

4. Findings

M01: Missing fungible asset metadata validation enables funding with wrong/worthless asset

Status: Resolved

Impact:

Likelihood:

Severity: Medium

Location: https://github.com/Lucid-Fi/pact-aptos-initial-audit/blob/6894b8cc0fa377c85050b07c575d0bb59b88c580/packages/core-loans/sources/loan_book.move#L1372

>_ loan_book.move

```
1370:     }
1371:
1372:     fun fund_loan_internal(
1373:         pending_loan_object: Object<PendingLoan>, fa: FungibleAsset, funder: address
1374:     ) acquires PendingLoan {
```

Description:

The **fund_loan_internal function** does not verify that the fungible asset (fa) provided by the funder matches the expected **pending_loan.fa_metadata**. This allows an attacker to fund a **PendingLoan** using a different, potentially worthless FA that merely matches the **principal_amount** in raw units. As long as **fungible_asset(&fa;)** equals the required amount, the call succeeds. When borrower calls **accept_loan**, it will internally call **start_loan::get_fa_for_loan()**, as there won't be assets at **pending_loan_address**, it will fallback and withdraw from the **loan_book** primary fungible store, even though the loan NFT will be transferred to the lender
- It should have an equality check between provided fa's metadata and **pending_loan.fa_metadata**.
- Due to this the Loan can be funded with an unintended/valueless asset.

Recommendation:

Enforce metadata check, abort unless **fa.metadata == pending_loan.fa_metadata**.

Developer Response:

Fixed according to the provided recommendation.
Change: https://github.com/Lucid-Fi/pact-aptos-initial-audit/pull/1

Note: Loans have all historically been offered, funded, and initiated in the same transaction. So all previous usecases are not exposed to this issue.

M02: Multiple funding allowed for pending loan leads to overwritten lender and stuck funds

Status: Resolved

Impact:

Likelihood:

Severity: Medium

Location: https://github.com/Lucid-Fi/pact-aptos-initial-audit/blob/6894b8cc0fa377c85050b07c575d0bb59b88c580/packages/core-loans/sources/loan_book.move#L1192

>_ loan_book.move

```
1190:     }
1191:
1192:     public fun fund_loan(
1193:         pending_loan_object: Object<PendingLoan>, funder: address, fa: FungibleAsset
1194:     ) acquires PendingLoan {
```

Description:

The **fund_loan_internal** function permits for multiple funding operations against the same PendingLoan without verifying whether a lender has already been assigned. Specifically, it lacks a check against **pending_loan.lender.is_some()**.
As a result:
- A subsequent **fund_loan** call can overwrite the existing lender on a pending loan.
- Funds from earlier funders and new funder become irrecoverable or “stuck” since. It will withdraw from **loan_book_address** as it will fallback into the **else_if** from the **if** case
- This creates both loss-of-funds risk for the previous lender's and **loan_book**. The new lender will get payment flows from borrower

Recommendation:

Add a guard in **fund_loan_internal**, abort if **pending_loan.lender.is_some()** to prevent multiple fundings.

Developer Response:

Fixed according to the provided recommendation.
Change: https://github.com/Lucid-Fi/pact-aptos-initial-audit/pull/1

M03: Griefing attack via unauthorized deposits to pending loan address

Status: Resolved

Impact:

Likelihood:

Severity: Medium

Location: https://github.com/Lucid-Fi/pact-aptos-initial-audit/blob/6894b8cc0fa377c85050b07c575d0bb59b88c580/packages/core-loans/sources/loan_book.move#L1526

>_ loan_book.move

```
1524:
1525:         if (primary_fungible_store::balance(loan_address, fa_metadata)
1526:             == pending_loan_principal) {
1527:             primary_fungible_store::withdraw(
1528:                 loan_signer, fa_metadata, pending_loan_principal
```

Description:

Because the balances are checked with strict equality, an attacker could deposit 1 wei into **loan_address** and force the if branch to fail and go over to the else, hence triggering the fallback mechanism.

Recommendation:

Check the balances using **>=** instead of **==**.

Developer Response:

Fixed according to the provided recommendation.
Change: https://github.com/Lucid-Fi/pact-aptos-initial-audit/pull/1

L01: Payment order bitmap allows ambiguous order

Status: Resolved

Impact: Low Medium High

Likelihood: Low Medium High

Severity: Low

Location: https://github.com/Lucid-Fi/pact-aptos-initial-audit/blob/6894b8cc0fa377c85050b07c575d0bb59b88c580/packages/core-loans/sources/payment_schedule_bitmap.move#L69

>_ payment_schedule_bitmap.move

```
67:     }
68:
69:     public fun pay_intervals_get_third(order_bitmap: u8): u8 {
70:         let first = pay_intervals_get_first(order_bitmap);
71:         let second = pay_intervals_get_second(order_bitmap);
```

Description:

In **payment_schedule_bitmap** module, users can call **pay_intervals_get_third()** with the **order_bitmap**. In the bitmap parameter, it is a **u8** type. In this, 0-1 bits represent the order of the PRINCIPAL, 2-3 bits represent the order of the INTEREST, and 4-5 bits represents the order of FEE.
If users provide order of Any two of (PRINCIPAL, INTEREST, FEE), as one and two, and for the remaining third one, even if the user provides the same ambiguous order of the before two, it will take the value of third one based on the first and second values

For example:
Bitmap : (00011001) -> This bitmap says PRINCIPAL First, INTEREST second, and FEE is also First (which should be incorrect), as FEE and PRINCIPAL has same order as First. But here it will by default take the FEE as third based on the earlier two values. The function doesn't validate that the bitmap is valid. It assumes the bitmap represents a valid and unambiguous payment order

When we have PRINCIPAL_FIRST | INTEREST_SECOND | FEE_FIRST:
- **pay_intervals_get_first()** will return PRINCIPAL (because it checks PRINCIPAL_FIRST first)
- **pay_intervals_get_second()** will return INTEREST (because it checks INTEREST_SECOND)
- **pay_intervals_get_third()** will return FEE (calculated as 6 - (2 + 3) = 1)
But this is logically incorrect because Fee was marked as "first priority" in the bitmap, not third

Although **payment_order_bitmap** will be passed by an admin in the **loan_book::offer_loan**, it would be better to validate it once, to avoid any confusion.

Proof of Concept:

RUST

```rust
#[test]
    fun test_ambigious_order_in_bitmap() {
        //011001 (principal first, interest second, fee first)
        let order = PRINCIPAL_FIRST | INTEREST_SECOND | FEE_FIRST;
```

... continued RUST

```
        let third = pay_intervals_get_third(order);
        assert!(third == FEE, EBITMAP_ORDER_WRONG)
    }
```

Recommendation:

The remaining type (third) should have neither FIRST nor SECOND bits set. This makes the bitmap more explicit and prevents confusion.

Developer Response:

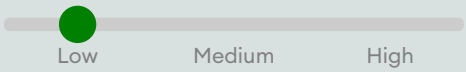
Fixed by adding validation for payment bitmap.
Change: https://github.com/Lucid-Fi/pact-aptos-initial-audit/pull/2/commits/ccfcee24d55373a1d5057694fda9426b3cf13bca

L02: Ungated Snapshot spam can bloat claim history

Status: Resolved

Impact:

Likelihood:



Severity: Low

Location: https://github.com/Lucid-Fi/pact-aptos-initial-audit/blob/6894b8cc0fa377c85050b07c575d0bb59b88c580/packages/token-utils/sources/passthrough_token.move#L299

>_ passthrough_token.move

```
297:     ) acquires PassThroughTokenState {
298:         assert!(
299:             supports_ungated_snapshot(token_state), EUNGATED_SNAPSHOT_NOT_SUPPORTED
300:         );
301:         snapshot(token_state);
```

Description:

If the **PassthroughToken** address has **UngatedSnapshot** resource, anyone can call **snapshot_ungated** without authorization. If an attacker calls it frequently, each call that finds non-zero **claimable_amount** will append a new entry to **claimable_history** for each pool. Over time, this makes **claimable_history** very large. Since **calculate_claimable_amount** iterates from a user's **next_claim_index** through the end of **claimable_history**, the per-claim cost grows with the number of snapshots. This can degrade performance and potentially lead to a DoS-like condition for users with many unclaimed snapshots.

Recommendation:

Gate **snapshot_ungated**, so that only admins in **PassThroughTokenState** can call this function.
During our meetings with Pact Engineering Team, they came up two additional valid ways to fix this:
- Leave it ungated but record a new snapshot only once per a configurable interval.
- Leave it ungated but record a new snapshot only when a configurable threshold of claimable tokens is reached.

Developer Response:

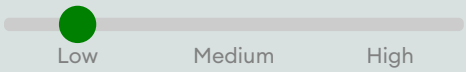
Fixed by adding configurable threshold for token snapshot.
Change: https://github.com/Lucid-Fi/pact-aptos-initial-audit/pull/2/commits/a13f5fc3a834203f48bfa9433f4cd46b2c9ae9d2

L03: DoS via unbounded vector growth and loops

Status: Resolved

Impact:

Likelihood:



Severity: Low

Location: https://github.com/Lucid-Fi/pact-aptos-initial-audit/blob/6894b8cc0fa377c85050b07c575d0bb59b88c580/packages/token-utils/sources/passthrough_token.move#L181-L182

>_ passthrough_token.move

```
179:
180:         let total_claimable_amount = 0u64;
181:         let i = (starting_index as u64);
182:         while (i < max_claim_index) {
183:             let claimable_per_token = *vector::borrow(&indexed_claim_history, i);
184:             let claimable_amount =
```

Description:

When a user claims payout for a token, it would have to iterate over the claimable amounts vector for that token. The iteration goes on from the last index where he claimed until the latest claim amount available. Provided by the fact that the claimable amount vector does not have bounds and therefore can grow indefinitely, if a user never has last claim index equal to 0 (never claimed) or a low value, and the vector is big, the operation will fail and there's no way of partially claiming.

Recommendation:

Add a way to partially claim, e.g., make a max number of iterations. Additionally, consider making the claim amounts a circular buffer so that it does not grow indefinitely.

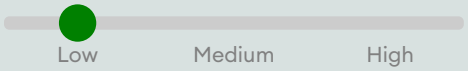
Developer Response:

Fixed by adding partial claim functionality.
Change: https://github.com/Lucid-Fi/pact-aptos-initial-audit/pull/2/commits/10b69698c1c4b4a0695529826f5f5a73efa3b666

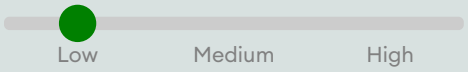
L04: Late fee receiver locked to Originator by default

Status: Resolved

Impact:



Likelihood:



Severity: Low

Location: https://github.com/Lucid-Fi/pact-aptos-initial-audit/blob/6894b8cc0fa377c85050b07c575d0bb59b88c580/packages/core-loans/sources/loan_book.move#L2202-L2208

>_ loan_book.move

```
2200:     }
2201:
2202:     inline fun get_late_fee_receiver(loan_address: address): &LateFeeReceiver {
2203:         if (exists<LateFeeReceiver>(loan_address)) {
2204:             borrow_global<LateFeeReceiver>(loan_address)
2205:         } else {
2206:             &LateFeeReceiver::Originator
2207:         }
2208:     }
2209:
2210:     #[test_only]
```

Description:

If no **LateFeeReceiver** resource is present under the loan address, **get_late_fee_receiver** function defaults to Originator. This module doesn't expose any setter function to set or change **LateFeeReceiver** for loans, so late fees always go to the originator.

Recommendation:

Add an authorised setter to publish/update **LateFeeReceiver** for loans. Also emit an event on change.

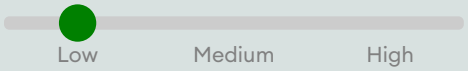
Developer Response:

Fixed by adding permissioned updates for late fee receiver.
Change: https://github.com/Lucid-Fi/pact-aptos-initial-audit/pull/2/commits/810aad27ba55b5b51882830d7301b9f9cfa2ec40

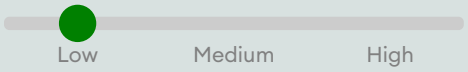
L05: Historical endpoints lack timestamp validation

Status: Resolved

Impact:



Likelihood:



Severity: Low

Location: https://github.com/Lucid-Fi/pact-aptos-initial-audit/blob/6894b8cc0fa377c85050b07c575d0bb59b88c580/packages/core-loans/sources/loan_book.move#L1240-L1248

>_ loan_book.move

```
1238:         timestamp_us: u64
1239:     ): PaymentReceipt acquires Loan, LoanBook, LoanContributionTracker, LateFeeRules,
          LateFeeTracker, LateFeeReceiver {
1240:         let loan = borrow_global<Loan>(object::object_address(&loan_object));
1241:         let loan_book_address = object::object_address(&loan.loan_book);
1242:
1243:         assert!(
1244:             loan_book_address == historical_loan_book_ref.self,
1245:             EINVALID_HISTORICAL_LOAN_BOOK_REF
1246:         );
1247:
1248:         pay_loan_internal(
1249:             loan_object,
1250:             fa,
```

Description:

Timestamp is left unchecked while the purpose of these functions is to fix situations from the past. The provided timestamp could theoretically be from the future. Also, by mistake, the timestamp could be sent as 0, case in which differences between relatively current timestamps minus provided timestamp could be huge and could lead to enormous differences in accumulated fees.

Recommendation:

Sanity check the timestamp arg, e.g., **loan.start_time_us <= timestamp_us <= now_microseconds()**

Developer Response:

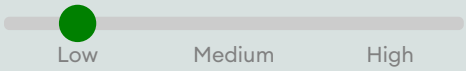
Fixed by verifying payment timestamp.
Change: https://github.com/Lucid-Fi/pact-aptos-initial-audit/pull/2/commits/84608e3fe5bb2064bee260c583810d84d9324ac6

L06: Incorrect verification of MAXIMUM_OBJECT_NESTING in ensure_owner::verify_descendant

Status: Resolved

Impact:

Likelihood:



Severity: Low

Location: https://github.com/Lucid-Fi/pact-aptos-initial-audit/blob/6894b8cc0fa377c85050b07c575d0bb59b88c580/packages/utils/sources/ensure_owner.move#L43

>_ ensure_owner.move

```
41:         );
42:
43:         let current_address = destination;
44:         let count = 0;
45:         while (owner != current_address) {
```

Description:

pact::ensure_owner::verify_descendant is meant to confirm that a destination object is ultimately owned by owner, enforcing a maximum chain depth of MAXIMUM_OBJECT_NESTING of 8. The function currently initializes current_address to destination and then walks up the chain. This is off by one relative to the intended semantics: it should begin at the owner of destination, not destination itself. This should only check up to Maximum Object Nesting of 7.

Aptos official logic handle this correctly here, by initially checking the owner with the owner of the destination object address, but not with the destination itself.

Proof of Concept:

RUST

```
#[test(owner = @test_admin)]
    #[expected_failure(abort_code = 131075)]
    fun test_max_object_ownership_nesting(owner: &signer) {
        // Creating multi-level nesting: owner -> obj1 -> obj2 -> obj3 -> obj4 -> obj5 -> obj6
     -> obj7 -> obj8
        let constructor_ref_1 = object::create_object(signer::address_of(owner));
        let obj1 = object::object_from_constructor_ref<ObjectCore>(&constructor_ref_1);
        let obj1_address = object::object_address(&obj1);

        let constructor_ref_2 = object::create_object(obj1_address);
        let obj2 = object::object_from_constructor_ref<ObjectCore>(&constructor_ref_2);
        let obj2_address = object::object_address(&obj2);

        let constructor_ref_3 = object::create_object(obj2_address);
        let obj3 = object::object_from_constructor_ref<ObjectCore>(&constructor_ref_3);
        let obj3_address = object::object_address(&obj3);
```

... continued RUST

```
        let constructor_ref_4 = object::create_object(obj3_address);
        let obj4 = object::object_from_constructor_ref<ObjectCore>(&constructor_ref_4);
        let obj4_address = object::object_address(&obj4);

        let constructor_ref_5 = object::create_object(obj4_address);
        let obj5 = object::object_from_constructor_ref<ObjectCore>(&constructor_ref_5);
        let obj5_address = object::object_address(&obj5);

        let constructor_ref_6 = object::create_object(obj5_address);
        let obj6 = object::object_from_constructor_ref<ObjectCore>(&constructor_ref_6);
        let obj6_address = object::object_address(&obj6);

        let constructor_ref_7 = object::create_object(obj6_address);
        let obj7 = object::object_from_constructor_ref<ObjectCore>(&constructor_ref_7);
        let obj7_address = object::object_address(&obj7);

        let constructor_ref_8 = object::create_object(obj7_address);
        let obj8 = object::object_from_constructor_ref<ObjectCore>(&constructor_ref_8);
        let obj8_address = object::object_address(&obj8);

        // `verify_descendant` doesn't work for the Maximum deepest level
        verify_descendant(signer::address_of(owner), obj8_address);
    }
```

Paste the above PoC in **ensure.move** file and run the PoC with the below command

SH

```
aptos move test --filter test_max_object_ownership_nesting
```

Recommendation:

Initialize **current_address** to the owner of object at **destination** address, instead of taking **destination** address directly as **current_address**.

DIFF

```
public fun verify_descendant(owner: address, destination: address) {
        assert!(
            object::is_object(destination),
            error::not_found(EOBJECT_DOES_NOT_EXIST)
        );

        let current_address = destination;
+       let object = object::address_to_object<ObjectCore>(current_address);
+       let current_address = object::owner<ObjectCore>(object);
        ....
}
```

During our meetings with Pact Engineering Team, they came up with an even better way to fixing this: use directly the Aptos function that does this.

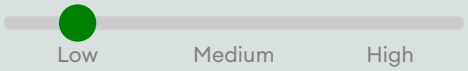
Developer Response:

Fixed by updating starting point for verify descendant.
Change: https://github.com/Lucid-Fi/pact-aptos-initial-audit/pull/2/commits/8d3947d7b3735f6db872e325528178e460bf071c

L07: Incorrect calculation of aggregates::numerator_time_weighted_value()

Status: Resolved

Impact:



Likelihood:

Severity: Low

Location: https://github.com/Lucid-Fi/pact-aptos-initial-audit/blob/6894b8cc0fa377c85050b07c575d0bb59b88c580/packages/utils/sources/aggregates.move#L62

>_ aggregates.move

```
60:         let numerator = aggregate.numerator as u128;
61:         let elapsed: u128 = current_time - last_timestamp;
62:         let current_weight: u128 = elapsed + last_value;
63:
64:         (current_weight + numerator) as u128
```

Description:

The **numerator_time_weighted_value()** should return the up-to-date numerator of a time-weighted average by adding the in-progress interval since last_timestamp as **elapsed * last_value** to the **numerator**. Instead, it uses **elapsed + last_value** , which is incorrect and inconsistent with **get_time_weighted_value()** and **update_time_weighted_value()**. This causes inflation when **elapsed == 0/1** and typically severe deflation for **elapsed >= 2**.

Recommendation:

Align **numerator_time_weighted_value()** with the module's time-weighted math and by using multiplication.

DIFF

```
-     let current_weight: u128 = elapsed + last_value;
+     let current_weight: u128 = elapsed * last_value;
```

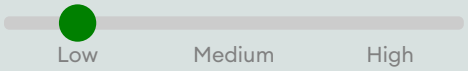
Developer Response:

Fixed time weighted math.
Change: https://github.com/Lucid-Fi/pact-aptos-initial-audit/pull/2/commits/974bc40f25d49f29b42220073fe3072f5ff34477

L08: Forward function lacks only owner check

Status: Resolved

Impact:



Likelihood:

Severity: Low

Location: https://github.com/Lucid-Fi/pact-aptos-initial-audit/blob/6894b8cc0fa377c85050b07c575d0bb59b88c580/packages/enrichers/sources/escrow.move#L36-L39

>_ escrow.move

```
34:         amount: option::Option<u64>
35:     ) acquires Escrow, ForwardingConfig {
36:         assert!(is_forwarding_enabled(escrow), EFORWARDING_NOT_ENABLED);
37:         let forwarding_config = borrow_global<ForwardingConfig>(object::object_address
        (&escrow));
38:         let owner = object::owner(escrow);
39:         let recipient = option::destroy_with_default(forwarding_config.forward_to, own
        er);
40:
41:         let escrow_signer = escrow_signer(&escrow);
```

Description:

The **forward** function is a public entry with no caller verification. If forwarding is enabled, any user can call it to transfer arbitrary or all assets from the escrow to the configured **forward_to** address (or owner if none). One could think of griefing scenarios where for example you call forward with 50% of the total amount (amount is a parameter) and you break settled legal flows, where one transaction hash would be permitted. Plus maybe the receiver was not set yet, case in which **forward** will forward to the owner of the escrow, which might not be desired.

Recommendation:

Add only owner checks as **claim** and **withdraw** have.

Developer Response:

Fixed by updating forward function to forward entirety of tokens.
Change: https://github.com/Lucid-Fi/pact-aptos-initial-audit/pull/2/commits/c99ad4bdd25af1b204792230cd8d4f344eba74f0

L09: PaymentCategorizerCreated event is not emitted correctly

Status: Resolved

Impact: Low Medium High

Likelihood: Low Medium High

Severity: Low

Location: https://github.com/Lucid-Fi/pact-aptos-initial-audit/blob/6894b8cc0fa377c85050b07c575d0bb59b88c580/packages/enrichers/sources/payment_flow.move#L28-L46

>_ payment_flow.move

```
26:     }
27:
28:     public fun categorize_future_payments(
29:         signer: &signer,
30:         principal_receiver: address,
31:         interest_receiver: address,
32:         fee_receiver: address
33:     ) {
34:         move_to(signer, PaymentCategorizer {
35:             principal: principal_receiver,
36:             interest: interest_receiver,
37:             fee: fee_receiver
38:         });
39:
40:         event::emit(PaymentCategorizerCreated {
41:             receiver: principal_receiver,
42:             principal: principal_receiver,
43:             interest: interest_receiver,
44:             fee: fee_receiver
45:         });
46:     }
47:
48:     public fun deposit(
```

Description:

In pact a receiver can categorize how the payments are distributed (how fees, interest and principal are transferred).

In the case, that the **receiver** (receiver address of the loan payment) is different than the address that the principal will be sent too, the event will be emitted incorrectly.

Recommendation:

The receiver should be the address of the receiver which could be different from the principal receiver. A possible fix could be like this.

RUST

```
public fun categorize_future_payments(
        signer: &signer,
```

... continued RUST

```
        principal_receiver: address,
        interest_receiver: address,
        fee_receiver: address
    ) {
        move_to(signer, PaymentCategorizer {
            principal: principal_receiver,
            interest: interest_receiver,
            fee: fee_receiver
        });
        let receiver = signer::address_of(signer);
        event::emit(PaymentCategorizerCreated {
            receiver: receiver,
            principal: principal_receiver,
            interest: interest_receiver,
            fee: fee_receiver
        });
```

Developer Response:

Fixed payment categorizer event.
Change: https://github.com/Lucid-Fi/pact-aptos-initial-audit/pull/2/commits/ba66a22c0b24d8737b49405dc90001ea0172bf7e

L10: Unchecked input in set_late_fee_rules could lead to division by zero

Status: Resolved

Impact: Low Medium High

Likelihood: Low Medium High

Severity: Low

Location: https://github.com/Lucid-Fi/pact-aptos-initial-audit/blob/6894b8cc0fa377c85050b07c575d0bb59b88c580/packages/core-loans/sources/loan_book.move#L682-L690

>_ loan_book.move

```
680:         admin: &signer, loan_book: Object<LoanBook>, rules: LateFeeRules
681:     ) acquires LoanBook {
682:         assert!(is_admin(loan_book, signer::address_of(admin)), ENOT_ADMIN);
683:         let loan_book_address = object::object_address(&loan_book);
684:         let loan_book = borrow_global_mut<LoanBook>(loan_book_address);
685:         let loan_book_signer = object::generate_signer_for_extending(&loan_book.extend
        _ref);
686:
687:         move_to(
688:             &loan_book_signer,
689:             rules
690:         );
691:
692:         event::emit(
```

Description:

set_late_fee_rules does not validate the input and could lead to division by 0 in delinquent_linear_accrual_late_fee

Recommendation:

Validate the input when setting fee rules

Developer Response:

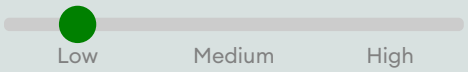
Fixed by adding validation for late fee denominator.
Change: https://github.com/Lucid-Fi/pact-aptos-initial-audit/pull/2/commits/3f71cf42c95de553f23c0ad3b9073c87ae46f6fd

L11: can_retire() is not implemented correctly leading to DoS in some cases

Status: Resolved

Impact:

Likelihood:



Severity: Low

Location: https://github.com/Lucid-Fi/pact-aptos-initial-audit/blob/6894b8cc0fa377c85050b07c575d0bb59b88c580/packages/core-loans/sources/loan_book.move#L390-L395

>_ loan_book.move

```
388:
389:     #[view]
390:     public fun can_retire(loan: Object<Loan>): bool acquires Loan {
391:         let loan = borrow_global<Loan>(object::object_address(&loan));
392:         let schedule_length = vector::length(&loan.payment_schedule);
393:         let last_installment = vector::borrow(&loan.payment_schedule, schedule_length
        - 1);
394:         last_installment.principal == 0 && last_installment.interest == 0 && last_inst
        allment.fee == 0
395:     }
396:
397:     #[view]
```

Description:

can_retire() only checks if the last installment is set to have zero principal, fee and interest. The loan book however doesn't have a restriction on the way installments are structured. An originator can set an installment with 0 fee, interest and principal. This will lead to **can_retire()** returning true instead of false.

Because **can_retire()** is used externally by the **hybrid_loan_book**. It is currently possible to DoS **repay()** on hybrid loan book by setting the last installment as 0 fee, principal and interest (**time_due_us** need to be set correctly)

Recommendation:

The simplest and most gas efficient way to fix this is to add an extra check for the **current_payment_index** in **can_retire()**

DIFF

```
public fun can_retire(loan: Object<Loan>): bool acquires Loan {
        let loan = borrow_global<Loan>(object::object_address(&loan));
        let schedule_length = vector::length(&loan.payment_schedule);
+       let current_payment_index = loan.current_payment_index as u64;
+       let is_last_installment = current_payment_index >= schedule_length -1;
        let last_installment = vector::borrow(&loan.payment_schedule, schedule_length - 1);
        last_installment.principal == 0 && last_installment.interest == 0 && last_installment.
    fee == 0
    }
```

Developer Response:

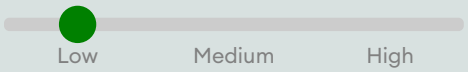
Fixed by checking current payment index.
Change: https://github.com/Lucid-Fi/pact-aptos-initial-audit/pull/2/commits/6ab0e723124804ce6f02223403cdab0502b698ef

L12: Missing Validation on offer_loan() can DoS/Grief Borrowing

Status: Resolved

Impact:

Likelihood:



Severity: Low

Location: https://github.com/Lucid-Fi/pact-aptos-initial-audit/blob/6894b8cc0fa377c85050b07c575d0bb59b88c580/packages/core-loans/sources/loan_book.move#L1542-L1557

>_ loan_book.move

```
1540:     fun validate_payment_schedule(
1541:         payment_schedule: &vector<Interval>,
1542:         start_time_us: u64
1543:     ) {
1544:         let intervals_len = vector::length(payment_schedule);
1545:         let i = 0;
1546:
1547:         while (i < intervals_len) {
1548:             let interval = vector::borrow(payment_schedule, i);
1549:             if (interval.time_due_us <= start_time_us) {
1550:                 assert!(interval.principal == 0, EPAYMENT_SCHEDULE_INVALID_PRINCIPAL_D
          UE_BEFORE_START_TIME);
1551:                 assert!(i == 0, EPAYMENT_SCHEDULE_INVALID_INTEREST_DUE_BEFORE_START_TI
          ME);
1552:             } else  {
1553:                 break;
1554:             };
1555:
1556:             i = i + 1;
1557:         }
1558:     }
1559:
```

Description:

The **payment_schedule** is only validated for continuity in the **offer_loan()** entry point. When accepting a loan, however the same **payment_schedule** will be validated using the **validate_payment_schedule()**. The **validate_payment_schedule()** will revert if any of the installments has a principal with zero value and their **time_due_us** is over due.

This could lead to a griefing vector, where an originator will create a **payment_installment** with overdue **time_due_us** in one of the installment and set the principal there to 0. This will cause accepting the pending loan to revert in **accept_loan()** -> **start_loan()** -> **record_loan()** -> **validate_payment_schedule()**

Recommendation:

To protect against this griefing vector, we recommend validating the **payment_schedule** at origination (**offer_loan()**) we need to validate the payment schedule using the **validate_payment_schedule()** but instead of using **start_time_us** as **time.now** we should use **time.now + safety_margin** (can be configured by loan book admin)

DIFF

```
fun create_loan_internal(
        offerer: address,
    ): ConstructorRef acquires LoanBook {
        # ...
        let object_signer = object::generate_signer(&nft_constructor_ref);

        ensure_time_due_strictly_increases(&payment_schedule);
+       let start_time = timestamp::now_microseconds() + SAFETY_INTERVAL;
+       validate_payment_schedule(&payment_schedule, start_time);
        move_to(
```

Developer Response:

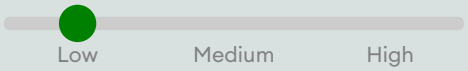
Fixed by using aggregate fields for checking payment schedule status rather than iterating every time.
Change: https://github.com/Lucid-Fi/pact-aptos-initial-audit/pull/2/commits/5b77dc0bec7ae62b676bf282606b771138a6312a

L13: Unrestricted payment schedule could lead to the creation of unrepayable loans

Status: Resolved

Impact:

Likelihood:



Severity: Low

Location: https://github.com/Lucid-Fi/pact-aptos-initial-audit/blob/6894b8cc0fa377c85050b07c575d0bb59b88c580/packages/core-loans/sources/loan_book.move#L1886-L1895

>_ loan_book.move

```
1884:         let payment_amount = fungible_asset::amount(&payment_fa);
1885:
1886:         assert!(
1887:             get_cur_debt_from_payment_schedule(&loan.payment_schedule)
1888:                 >= payment_amount,
1889:             EREAPYMENT_TOO_HIGH
1890:         );
1891:
1892:         let (toward_fees, toward_interest, toward_principal, next_due_index) =
1893:             pay_intervals(
1894:                 &mut loan.payment_schedule, payment_amount, payment_order_bitmap, loan
          .current_payment_index
1895:             );
1896:
1897:         loan.current_payment_index = next_due_index;
```

Description:

When offering a loan, the system doesn't restrict the length of the payment schedule vector, meaning the originator of the loan can create a loan with a very large payment_schedule that will not fail in **offer_loan()** and **fund_loan()**. But **accept_loan()** and/or **repay_loan()** will fail because they loop through the payment schedule multiple times. For e.g., in **offerLoan()** we only loop once through the **payment_interval** when doing the **ensure_time_due_strictly_increases()** validation check. However in **repay_loan()** and **accept_loan()** we will loop through the **payment_schedule** multiple times, e.g., functions that are used in those functions that implement loops: **get_cur_debt_from_payment_schedule()**, **pay_intervals()**, **get_remaining_principal()**, **get_fa_for_loan()**, **get_initial_fa_amount()**, **record_loan()**.

Proof of Concept:

A simplified scenario:
- Originator offers a loan with a large payment_schedule(), **offer_loan()** will succeed because we only loop once
- Funder calls **fundLoan()** on the generated **PendingLoan()**. This will also succeed because we only loop once through the vector (**get_remaining_principal()** is called to compute funding amount)
- Now when the borrower, tries to accept the loan this will fail either because of out of gas or a very large transaction size. As the **accept_loan()** function will do multiple loops through the payment schedule.

This will lead to either the borrower not being able to start the loan or not being able to repay it.

Recommendation:

When offering a loan, we need to validate the length of the **payment_schedule** vector. We recommend that the team adds a plausible length restriction (depending on Use Case of system) on the **payment_schedule** to mitigate any DoS for large **payment_schedule** vectors.

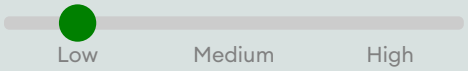
Developer Response:

Fixed by setting max payment schedule length to recommended 2000.
Change: https://github.com/Lucid-Fi/pact-aptos-initial-audit/pull/2/commits/664501a67c8a9b1463847c044645821e5f13d6bc

L14: Anyone can manipulate the loan.payment_count with 0 cost

Status: Resolved

Impact:



Likelihood:

Severity: Low

Location: https://github.com/Lucid-Fi/pact-aptos-initial-audit/blob/6894b8cc0fa377c85050b07c575d0bb59b88c580/packages/core-loans/sources/loan_book.move#L1943

>_ loan_book.move

```
1941:         );
1942:
1943:         loan.payment_count = loan.payment_count + 1;
1944:         try_update_contribution_tracker(
1945:             loan_address,
```

Description:

The system allows anyone (permissionless) to make repayments with 0 assets. The **loan.payment_count** tracker will however be unconditionally incremented everytime the **pay_loan_internal()** is invoked. Meaning anyone can call repay with 0 assets multiple times to manipulate the tracker the way he want.

Proof of Concept:

We add the following **test_can_repay_with_zero_fa()** to showcase that repayment with 0 assets is possible.

RUST

```
fun test_can_repay_with_zero_fa(
        admin: &signer,
        originator: &signer,
        borrower: &signer,
        aptos_framework: &signer
    ) acquires LoanBook, PendingLoan, Loan, LoanContributionTracker, LateFeeRules, LateFeeTrac
    ker, LateFeeReceiver {
        setup_clock(aptos_framework);
        let (loan, fa_metadata, mint_ref) =
            setup_loan_test(
                admin,
                originator,
                borrower,
                signer::address_of(admin)
            );
        let loan_address = object::object_address(&loan);

        // Get initial state
        let initial_tracker = borrow_global<LoanContributionTracker>(loan_address);
        let initial_total_paid = initial_tracker.total_paid;
        let initial_principal_paid = initial_tracker.principal_paid;
```

... continued RUST

```
        let initial_interest_paid = initial_tracker.interest_paid;
        let initial_fees_paid = initial_tracker.fees_paid;

        // Try to repay with 0 amount
        let zero_fa = fungible_asset::mint(&mint_ref, 0);
        let receipt = repay_loan(loan, zero_fa);
        destroy_payment_receipt(receipt);

        // Check that nothing changed
        let final_tracker = borrow_global<LoanContributionTracker>(loan_address);
        assert!(final_tracker.total_paid == initial_total_paid, 0);
        assert!(final_tracker.principal_paid == initial_principal_paid, 1);
        assert!(final_tracker.interest_paid == initial_interest_paid, 2);
        assert!(final_tracker.fees_paid == initial_fees_paid, 3);
    }
```

Recommendation:

DIFF

```
fun pay_loan_internal(
        loan_object: Object<Loan>,
        fa: FungibleAsset,
        timestamp_us: u64,
        payment_order_bitmap: u8
    ): PaymentReceipt acquires LoanBook, Loan, LoanContributionTracker, LateFeeRules,
    LateFeeTracker, LateFeeReceiver {
        let loan_address = object::object_address(&loan_object);
        let loan = borrow_global_mut<Loan>(loan_address);
        let fa_metadata = fungible_asset::metadata_from_asset(&fa);
+   assert!(fungible_asset::amount(&fa) > 0, ERROR_AMOUNT_CANNOT_BE_ZERO);
```

Developer Response:

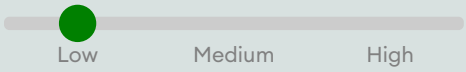
Fix by requiring payment amount to be greater than 0.
Change: https://github.com/Lucid-Fi/pact-aptos-initial-audit/pull/2/commits/bbe3a51ee6ac1083347af2c96ca33f8b9fa8f8ca

L15: Attackers could grief last payment of the loan

Status: Resolved

Impact:

Likelihood:



Severity: Low

Location: https://github.com/Lucid-Fi/pact-aptos-initial-audit/blob/6894b8cc0fa377c85050b07c575d0bb59b88c580/packages/core-loans/sources/loan_book.move#L1886-L1890

>_ loan_book.move

```
1884:         let payment_amount = fungible_asset::amount(&payment_fa);
1885:
1886:         assert!(
1887:             get_cur_debt_from_payment_schedule(&loan.payment_schedule)
1888:                 >= payment_amount,
1889:             EREAPYMENT_TOO_HIGH
1890:         );
1891:
1892:         let (toward_fees, toward_interest, toward_principal, next_due_index) =
```

Description:

It is possible to grief/block the last payment of a borrower on the loan book by frontruninng his repay transaction and repaying just 1 wei of tokens. This is possible because the **pay_loan_internal()** revert on overpayment instead of refunding any excess fees.

Proof of Concept:

An example scenario will look like this:
- Borrower has 50 Usdc left in his loan
- Borrower calls repay with 50 Usdc
- Attacker frontruns the transaction and repays the loan with 1 wei Usdc
- Borrower repay transaction executes after the attacker and aborts with **EREAPYMENT_TOO_HIGH**

Recommendation:

To mitigate this issue, we recommend reimbursing any excess funds submitted in the repayment instead of aborting.

Developer Response:

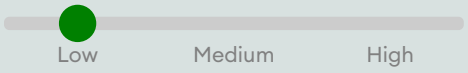
Fixed by removing exact payment check and return excess payment to caller.
Change: https://github.com/Lucid-Fi/pact-aptos-initial-audit/pull/2/commits/5c9f29056af98b20a3d57ba4a565aefe470031f1

L16: Possible spoofing in create_unnamed_escrow and create_unnamed_escrow_account

Status: Resolved

Impact:

Likelihood:



Severity: Low

Location: https://github.com/Lucid-Fi/pact-aptos-initial-audit/blob/6894b8cc0fa377c85050b07c575d0bb59b88c580/packages/enrichers/sources/escrow.move#L58-L66

>_ escrow.move

```
56:     }
57:
58:     public fun create_unnamed_escrow(owner_address: address): ConstructorRef {
59:         let constructor_ref = object::create_object(owner_address);
60:         let escrow_signer = object::generate_signer(&constructor_ref);
61:         let extend_ref = object::generate_extend_ref(&constructor_ref);
62:
63:         move_to(&escrow_signer, Escrow { extend_ref });
64:
65:         constructor_ref
66:     }
67:
68:     public fun create_unnamed_escrow_account(owner_address: address): Object<Escrow> {
```

Description:

The escrow module's **create_unnamed_escrow** function allows anyone to pass an arbitrary address to create an escrow object, returning a **ConstructorRef**. An attacker can front-run a victim's escrow creation, create a rogue escrow with the victim's address as the owner, and retain the **ConstructorRef** to forge a signer. If funds are deposited to this rogue escrow (e.g., via social engineering), the attacker can withdraw them using the forged signer, risking asset theft. While object addresses are unique due to **object::create_object** using the creator's address and transaction-derived creation number (preventing address collisions), the attack remains viable if deposits are misdirected to the attacker's escrow object, violating access control principles.

Recommendation:

Modify **create_unnamed_escrow** to require a signer parameter, ensuring only the owner can create their escrow:

RUST

```
public fun create_unnamed_escrow(signer: &signer): ConstructorRef {
    let owner_address = signer::address_of(signer);
    // ... rest as is
}
```

Developer Response:

Fix by updating escrow creation to return protected constructor ref.
Change: https://github.com/Lucid-Fi/pact-aptos-initial-audit/pull/2/commits/10f347e34c32a172b1b0d6189ee559e83b056866

5. Enhancement Opportunities

E01: Redundant whitelist object ownership check in whitelist::bulk_toggle

Location: https://github.com/Lucid-Fi/pact-aptos-initial-audit/blob/6894b8cc0fa377c85050b07c575d0bb59b88c580/packages/utils/sources/whitelist.move#L85

>_ whitelist.move

```
83:
84:     #[lint::skip(needless_mutable_reference)]
85:     public entry fun bulk_toggle(
86:         signer: &signer,
87:         whitelist_obj: Object<BasicWhitelist>,
```

Description:

The **whitelist::bulk_toggle** function updates the whitelist status for multiple addresses at once. It first checks the ownership of the whitelist object against the signer, then calls the **toggle** function for each address in a loop. However, **toggle** also performs this ownership check, making it redundant. It would be better to update the whitelist status for all addresses directly within **bulk_toggle**.

Potential Benefit:

Directly updating the whitelist status within the **bulk_toggle** eliminates duplicate object ownership checks inside each **toggle** call during loops. Also reduces gas costs by eliminating redundant checks.

Recommendation:

Perform one ownership check at the start of **bulk_toggle**, then directly update the whitelist status for all the addresses within the **bulk_toggle** itself.

E02: Duplicate error code values creating ambiguous error handling

Location: https://github.com/Lucid-Fi/pact-aptos-initial-audit/blob/6894b8cc0fa377c85050b07c575d0bb59b88c580/packages/core-loans/sources/payment_schedule_bitmap.move#L2-L3

>_ payment_schedule_bitmap.move

```
1: module pact::payment_schedule_bitmap {
2:     const EINVALID_ORDER_BITMAP: u64 = 444;
3:     const EINVALID_BITMAP_PRIORITY: u64 = 444;
4:
5:     const FEE: u8 = 1;
```

Description:

In **payment_schedule_bitmap** module, error codes **EINVALID_ORDER_BITMAP** and **EINVALID_BITMAP_PRIORITY** are assigned the same value **444**.
Similarly in **loan_book** module, the error codes **EINTERVALS_DONT_MATCH** and **EINDEX_DONT_MATCH** are assigned the same value **207**.

These duplicate error codes create ambiguity in error handling and debugging processes and complicates the process of identifying the specific cause for a transaction failure.

Potential Benefit:

Allocating a distinct error code for each error type helps to enhance clarity and facilitate precise error identification.

Recommendation:

Assign unique error codes to each error constant. This will make debugging easier and helps to precisely identify the actual error.

E03: Zero fee_numerator entries in FeeCollection are redundant

Location: https://github.com/Lucid-Fi/pact-aptos-initial-audit/blob/6894b8cc0fa377c85050b07c575d0bb59b88c580/packages/core-loans/sources/fee_manager.move#L153

>_ fee_manager.move

```
151:     }
152:
153:     public entry fun add_fee(
154:         signer: &signer,
155:         collection: Object<FeeCollection>,
```

Description:

The **fee_manager::add_fee** functions allows adding a **Fee** with **fee_numerator == 0**. These entries are stored and can be toggled, but they never affect disbursements because **disburse_fees** only processes fees where **fee.active && fee_numerator > 0**.

Potential Benefit:

Only adding Fee entries with **fee_numerator > 0** helps prevent **FeeCollection.fee** from growing unnecessarily large, avoids adding Fees that don't affect disbursements and emitting redundant Event **FeeAdded**.

Recommendation:

Add a simple check to block zero **fee_numerator** additions. Optionally enforce the same in **create_fee**.

RUST

```
assert!(fee_numerator > 0, EFEE_INVALID);
```

E04: Missing event emission in remove_late_fee_rules

Location: https://github.com/Lucid-Fi/pact-aptos-initial-audit/blob/6894b8cc0fa377c85050b07c575d0bb59b88c580/packages/core-loans/sources/loan_book.move#L671-L677

>_ loan_book.move

```
669:     }
670:
671:     public entry fun remove_late_fee_rules(
672:         admin: &signer, loan_book: Object<LoanBook>
673:     ) acquires LoanBook, LateFeeRules {
674:         assert!(is_admin(loan_book, signer::address_of(admin)), ENOT_ADMIN);
675:         let loan_book_address = object::object_address(&loan_book);
676:         move_from<LateFeeRules>(loan_book_address);
677:     }
678:
679:     public fun set_late_fee_rules(
```

Description:

Calling **set_late_fee_rules** moves the resource **LateFeeRules** resource to the loan book object and emits **LateFeeRulesUpdated**. Calling **remove_late_fee_rules** removes the same resource but doesn't emit any event. It's an inconsistency, state changes in one direction are visible to off-chain, the inverse change is not.

Potential Benefit:

Emitting a removal event makes off-chain tracking reliable and symmetric.

Recommendation:

Add a dedicated removal event to **remove_late_fee_rules** function.

E05: Redundant should_validate_due_date_continuity check

Location: https://github.com/Lucid-Fi/pact-aptos-initial-audit/blob/6894b8cc0fa377c85050b07c575d0bb59b88c580/packages/core-loans/sources/loan_book.move#L983

>_ loan_book.move

```
981:         };
982:
983:         if (should_validate_due_date_continuity(loan.loan_book)) {
984:             ensure_time_due_strictly_increases(&loan.payment_schedule);
985:         };
```

Description:

The module has a **PaymentScheduleUpdateValidationSettings** resource with two flags: **validate_principal_continuity** and **validate_due_date_continuity**. There's an entry function to toggle principal continuity (**toggle_payment_schedule_principal_validation**). Due-date continuity defaults to true on first write and is checked via **should_validate_due_date_continuity**, but cannot be turned off.

Potential Benefit:

If this **due_date_continuity** enforcement is meant to be permanent, removing the flag(**should_validate_due_date_continuity**) and directly validating **due_date** simplifies code and storage.

Recommendation:

If it is intended to always keep the **due_date_continuity** as **true**, remove the function **should_validate_due_date_continuity** and by default validate the due date.

E06: Missing event emission in set_deferred_receiver

Location: https://github.com/Lucid-Fi/pact-aptos-initial-audit/blob/6894b8cc0fa377c85050b07c575d0bb59b88c580/packages/core-loans/sources/nft_manager.move#L222-L226

>_ nft_manager.move

```
220:         signer: &signer, token: Object<RwaTokenConfig>, deferred_receiver: address
221:     ) acquires RwaTokenConfig {
222:         assert!(object::owner(token) == signer::address_of(signer), ENOT_OWNER_OF_TOKE
         N);
223:
224:         let token_address = object::object_address(&token);
225:         let token_config = borrow_global_mut<RwaTokenConfig>(token_address);
226:         token_config.deferred_receiver = option::some(deferred_receiver);
227:     }
228:
```

Description:

The **set_deferred_receiver** function does not issue an event which might be useful later.

Potential Benefit:

Improve transparency and traceability.

Recommendation:

Add an event to the endpoint.

E07: Add event emission in whitelist::toggle

Location: https://github.com/Lucid-Fi/pact-aptos-initial-audit/blob/6894b8cc0fa377c85050b07c575d0bb59b88c580/packages/utils/sources/whitelist.move#L74-L82

>_ whitelist.move

```
72:         new_address: address,
73:         status: bool
74:     ) acquires BasicWhitelist {
75:         ensure_owner::verify_descendant(
76:             signer::address_of(signer), object::object_address(&whitelist_obj)
77:         );
78:         let whitelist =
79:             borrow_global_mut<BasicWhitelist>(object::object_address(&whitelist_obj));
80:
81:         smart_table::upsert(&mut whitelist.whitelist, new_address, status);
82:     }
83:
84:     #[lint::skip(needless_mutable_reference)]
```

Description:

The **pact::whitelist::toggle** function modifies whitelist status but does not emit the **WhitelistStatusUpdated** event.

Potential Benefit:

This could facilitate off-chain tracking and monitoring.

Recommendation:

Consider adding **WhitelistStatusUpdated** event emission in the **toggle** function after the state update.

E08: Removing redundant check will improve gas efficiency

Location: https://github.com/Lucid-Fi/pact-aptos-initial-audit/blob/6894b8cc0fa377c85050b07c575d0bb59b88c580/packages/enrichers/sources/escrow.move#L93-L93

>_ escrow.move

```
91:         signer: &signer, escrow: &Object<Escrow>, metadata: Object<T>
92:     ): FungibleAsset acquires Escrow {
93:         ensure_owner(signer, escrow);
94:         let escrow_balance =
95:             primary_fungible_store::balance(object::object_address(escrow), metadata);
```

Description:

Ensure_owner is redundant in **withdraw_all()** as it is already checked in **withdraw_from()**

RUST

```
public fun withdraw_from<T: key>(
        signer: &signer,
        escrow: &Object<Escrow>,
        metadata: Object<T>,
        amount: u64
    ): FungibleAsset acquires Escrow {
@>>     ensure_owner(signer, escrow);
        let escrow_signer = escrow_signer(escrow);
        let fa = primary_fungible_store::withdraw(&escrow_signer, metadata, amount);
        fa
    }
```

Potential Benefit:

Removing the redundant check helps improve the gas efficiency of the escrow module.

Recommendation:

Remove ensure owner from **withdraw_all()** as it is already checked in **withdraw_from()**

RUST

```
public fun withdraw_all<T: key>(
        signer: &signer, escrow: &Object<Escrow>, metadata: Object<T>
    ): FungibleAsset acquires Escrow {
@>>     ensure_owner(signer, escrow);
        let escrow_balance =
            primary_fungible_store::balance(object::object_address(escrow), metadata);
        withdraw_from(signer, escrow, metadata, escrow_balance)
    }
```

E09: Adjusting get_payment_schedule_summary_internal() will improve gas efficiency

Location: https://github.com/Lucid-Fi/pact-aptos-initial-audit/blob/6894b8cc0fa377c85050b07c575d0bb59b88c580/packages/core-loans/sources/loan_book.move#L716C1-L733C6

>_ loan_book.move

```
714:     }
715:
716:     fun get_payment_schedule_summary_internal(
717:         payment_schedule: &vector<Interval>
718:     ): (u64, u64, u64) {
```

Description:

The **get_payment_schedule_summary_internal()** loops through the whole payment_schedule to calculate the **principal_sum**, **interest_sum** and **fee_sum**. Unlike the other functions that loop through the whole **payment_schedule** vector. However, every time a user pays off an installment the principal, interest and fee will be set to zero. This is tracked in **Loan.current_payment_index**.

Potential Benefit:

The **get_payment_schedule_summary_internal()** get called multiple times during functions that is called often (like **repay_loan()**). Starting from current debt index will save up to **length(payment_schedule)** iterations per **get_payment_schedule_summary_internal()** call.

Recommendation:

Add a mechanism to withdraw the fees:

E10: Unused DocumentSummaryAdded event

Location: https://github.com/Lucid-Fi/pact-aptos-initial-audit/blob/6894b8cc0fa377c85050b07c575d0bb59b88c580/packages/enrichers/sources/document_manager.move#L50-L54

>_ document_manager.move

```
48:     }
49:
50:     #[event]
51:     struct DocumentSummaryAdded has drop, store {
52:         collection: Object<DocumentCollection>,
53:         summary: DocumentSummary
54:     }
55:
56:     #[event]
```

Description:

The **document_manager.move** file includes the declaration of **DocumentSummaryAdded** event but it is left unused.

Potential Benefit:

Emitting the event might fix offchain flows that are expecting the event.

Recommendation:

Delete the event if the other silimar named event (**DocumentSummaryAddedGeneric**) is used instead. Or if this is unintended, use the event in the right functions (e.g., **add_document()**).

About Us

About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in web3 audits.

Built by a mix of experienced professionals in traditional enterprise and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team's background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, and key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members having publications with over 1,100 academic citations and trusted by projects securing over $500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the web3 space.

Projects may optionally request specific contributors to be part of their audit.

Audit Methodology

Our audit methodology is specialized to provide thorough security assessments of Aptos Move frameworks and modules. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure every Move package and entry function operates securely and as intended.

1. Program Context and Architecture Analysis

Our auditors begin by deeply examining your Aptos Move package documentation, intended functionality, and resource design. We meticulously map out module interactions, transactional entry points, and global storage layout. Special attention is given to understanding how your code interfaces with core Aptos framework modules, such as **aptos_framework::coin**, **aptos_framework::account**, and **aptos_framework::timestamp**. We also review integrations with external Move packages and ensure all abilities, arguments, and return values are validated correctly.

2. Threat Modeling

We conduct targeted threat modeling tailored specifically for Aptos' Move VM and resource-oriented execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from Move-specific issues, including but not limited to:

- Unauthorized resource publication or key account access
- Improper signer capability or **&signer;** usage
- Misconfigured **friend** declarations and module visibility
- Unsafe global storage writes or missing resource invariants
- Failure to enforce ability constraints (**key**, **store**, **drop**, **copy**)
- Risks stemming from reconfiguration hooks, event emission, or aggregator usage

3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your Move modules. This involves a comprehensive line-by-line inspection of source code, focusing on common Aptos vulnerabilities including, but not limited to:

- Missing or insufficient signer and address authorization checks
- Incorrect handling of capability storage, delegation, or revocation
- Unsafe use of table/aggregator APIs or unchecked resource destruction
- Arithmetic errors in on-chain invariants or interest calculations
- Incorrect event sequencing or metadata emissions
- Logic flaws in **move_to**/**move_from** flows or resource initialization
- Missing abort conditions on critical preconditions and edge-case validation
- Potential denial-of-service vectors tied to gas budgeting, storage growth, or reconfiguration

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited we may also include a detailed proof-of-concept exploit including code snippets and instructions on how the exploit could be performed.

4. Detailed Fix Review and Validation

After the initial audit and your team's subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with Aptos Move best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the Aptos Move ecosystem.

Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, expressed or implied, in connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.

Appendix A: Appendix: Coverage

The Move coverage reports across the project components demonstrate strong test thoroughness, with overall scores exceeding 89% in utils (89.73%) and core-loans (89.53%), and an excellent 94.27% in enrichers. Module-level coverage is consistently solid (80%+), indicating robust unit testing that minimizes untested risks-great work toward reliable, production-ready modules.

Core-loans package:

```tex
+-------------------------+
| Move Coverage Summary |
+-------------------------+
Module 0000000000000000000000000000000000000000000000000000000000000fff::fee_manager
>>> % Module coverage: 94.94
Module 0000000000000000000000000000000000000000000000000000000000000fff::payment_schedule_bitm
    ap
>>> % Module coverage: 89.69
Module 0000000000000000000000000000000000000000000000000000000000000fff::nft_manager
>>> % Module coverage: 92.51
Module 0000000000000000000000000000000000000000000000000000000000000fff::loan_book
>>> % Module coverage: 88.75
+-------------------------+
| % Move Coverage: 89.53 |
+-------------------------+
```

Enrichers package:

```tex
+-------------------------+
| Move Coverage Summary |
+-------------------------+
Module 0000000000000000000000000000000000000000000000000000000000000fff::custodied_account
>>> % Module coverage: 82.14
Module 0000000000000000000000000000000000000000000000000000000000000fff::document_manager
>>> % Module coverage: 96.98
Module 0000000000000000000000000000000000000000000000000000000000000fff::escrow
>>> % Module coverage: 95.15
Module 0000000000000000000000000000000000000000000000000000000000000fff::payment_director
>>> % Module coverage: 96.15
Module 0000000000000000000000000000000000000000000000000000000000000fff::risk_score_manager
>>> % Module coverage: 90.67
+-------------------------+
| % Move Coverage: 94.27 |
+-------------------------+
```

Utils package:

```tex
+-------------------------+
| Move Coverage Summary |
+-------------------------+
Module 0000000000000000000000000000000000000000000000000000000000000fff::aggregates
>>> % Module coverage: 93.15
```

... continued TEX

```
Module 0000000000000000000000000000000000000000000000000000000000000000fff::circular_vector
>>> % Module coverage: 92.04
Module 0000000000000000000000000000000000000000000000000000000000000000fff::date_time
>>> % Module coverage: 93.62
Module 0000000000000000000000000000000000000000000000000000000000000000fff::ensure_owner
>>> % Module coverage: 87.39
Module 0000000000000000000000000000000000000000000000000000000000000000fff::time_series_tracker
>>> % Module coverage: 82.22
Module 0000000000000000000000000000000000000000000000000000000000000000fff::whitelist
>>> % Module coverage: 84.29
Module 0000000000000000000000000000000000000000000000000000000000000000fff::utils
>>> % Module coverage: 80.43
Module 0000000000000000000000000000000000000000000000000000000000000000fff::xirr
>>> % Module coverage: 91.64
+-------------------------+
| % Move Coverage: 89.73 |
+-------------------------+
```